

Tutorial 3, Friday, 16/10/2026, by vrm

Today: *Languages, simulation, and logical abstraction*

Conventions. For an action-labelled LTS, let

$$\mathcal{L}(s) = \{w \in \Sigma^* : s \xrightarrow{w} s' \text{ for some } s'\}$$

be its language of all finite traces, including ε . For a Kripke structure, $\text{Tr}(s)$ denotes its set of infinite words of state labels; the transition relation is total. LTL uses universal path semantics. Write $s \leq t$ when t simulates s ; on Kripke structures, simulations preserve state labels exactly. HML uses action modalities on LTSs, while CTL uses state propositions on Kripke structures.

Scope. Give explicit witnesses and short arguments. Do not compute maximal bisimulation relations or use partition refinement. Exercises marked $\star$ are optional extensions.

A. What does an abstraction preserve?

Exercise 1. [*Equal languages, different branching*] Construct two finite pointed LTSs (T, s) and (U, t) such that $\mathcal{L}(s) = \mathcal{L}(t)$ but $s \not\sim t$.

- (a) Give an HML formula true at one initial state and false at the other.

Exercise 2. [*Bisimilar but not isomorphic*] Construct two finite pointed LTSs with different numbers of reachable states whose initial states are bisimilar. Exhibit a bisimulation witnessing this and explain why the systems are not isomorphic.

Exercise 3. [*Simulation versus language inclusion*]

- (a) Prove that $s \leq t$ implies $\mathcal{L}(s) \subseteq \mathcal{L}(t)$.
- (b) Construct finite LTSs showing that the converse fails. Explain the failure by identifying a transition that cannot be matched by a simulation.
- (c) Prove that, for deterministic LTSs, the converse holds. Here deterministic means that each state has at most one successor for each action.

Exercise 4. [*CTL fragments and preservation under simulation*] Assume $s \leq t$ in a Kripke structure. For each formula below, determine whether the following implications hold for all such pairs:

$$(\rightarrow) \quad s \models \varphi \Rightarrow t \models \varphi, \quad (\leftarrow) \quad t \models \varphi \Rightarrow s \models \varphi.$$

Answer *both*, *only* $\rightarrow$, *only* $\leftarrow$, or *neither*.

- | | | |
|-----------------------|---------------------------------------|--------------------------------------|
| (i) $p \wedge \neg q$ | (iv) $\text{EF}(p \wedge \text{EX}q)$ | (vii) $\text{EX}p \wedge \text{AX}q$ |
| (ii) $\text{EX}p$ | (v) $\text{AG}(p \vee \text{AF}q)$ | (viii) $\neg \text{EX}\neg p$ |
| (iii) $\text{AX}p$ | (vi) $\text{AGEF}p$ | (ix) $\text{EG}p \vee \text{AF}q$ |

Justify valid directions using preservation results. For each invalid direction, give a finite counterexample and a witnessing simulation.

Exercise 5. [*Trace equivalence and a distinguishing CTL formula*] Consider the following two Kripke structures. Unlisted transitions are absent.

- K : $s \rightarrow x, x \rightarrow u, x \rightarrow v, u \rightarrow u, v \rightarrow v$;
 $L(s) = L(x) = \emptyset, L(u) = \{p\}, L(v) = \{q\}$.
- H : $t \rightarrow y, t \rightarrow z, y \rightarrow u', z \rightarrow v', u' \rightarrow u', v' \rightarrow v'$;
 $L(t) = L(y) = L(z) = \emptyset, L(u') = \{p\}, L(v') = \{q\}$.

- (a) Show that $\text{Tr}(s) = \text{Tr}(t)$ and conclude that s and t satisfy the same LTL formulas.
- (b) Find a CTL formula distinguishing s from t . Your formula should express that one immediate successor offers two different possible futures.

Exercise 6. [*Choose a sufficient abstraction*] For each verification task, choose a sufficient relation between a concrete state c and an abstract state a . State the direction of any trace inclusion or simulation, and justify your answer in one sentence.

- (a) Transfer the LTL formula Fp from a to c .
- (b) Transfer the CTL formula AFp from a to c . How does this task relate to (a)?
- (c) Transfer the CTL formula EFp from c to a .
- (d) Preserve the truth of the CTL formula $EFAGp$ in both directions.

Use trace inclusion/equality, simulation, or bisimulation as appropriate. In (a)–(c), consider whether trace inclusion alone suffices; your relation need only be sufficient, so do not claim necessity without proof.

B. Language problems for finite and infinite-state systems

Acceptance conventions. Write $L(A)$ for an *accepted* finite-word language, distinguished from the all-traces language $\mathcal{L}$ above. NFAs and PDAs accept by final state after consuming the whole input; PDAs may have ε -moves. For the process and counter models below, every transition consumes one letter; there are no silent transitions.

- A **BPA** has finitely many rules $X \xrightarrow{a} \alpha$, where α is a word of variables. Its configurations are words, with $X\beta \xrightarrow{a} \alpha\beta$. An initial word is specified; acceptance is by reaching the empty word.
- A **BPP** has the same rule form but configurations are multisets of variables. A rule replaces one occurrence of X by the multiset α . An initial multiset is specified; acceptance is by reaching the empty multiset.
- A **VASS** has configurations $(q, \mathbf{v})$ with $\mathbf{v} \in \mathbb{N}^d$ and transitions $q \xrightarrow{a, \mathbf{z}} q'$, enabled when $\mathbf{v} + \mathbf{z} \geq \mathbf{0}$. An initial configuration and final control states are specified; final counter values are unrestricted. A deterministic VASS has at most one outgoing transition per control state and letter.

Exercise 7. [*NFA inclusion and equivalence*]

- (a) Give algorithms deciding $L(A) \subseteq L(B)$ and $L(A) = L(B)$ for two NFAs. Explain why simply interchanging final and non-final states of B does not compute its complement.
- (b) Show how to extract a distinguishing word when equivalence fails.
- (c) Give an upper bound on the length of a shortest distinguishing word in terms of the numbers of states of A and B .

Exercise 8. [*PDA languages and regular specifications*] Let P be a PDA and A an NFA over the same alphabet.

- (a) Show that $L(P) \subseteq L(A)$ is decidable using closure properties and an emptiness test.
- (b) Investigate whether $L(A) \subseteq L(P)$ is decidable. Consider the special case $L(A) = \Sigma^*$. (Show that halting problem for automaton with two pushdowns is undecidable. Try to accept all incorrect runs of an automaton with two pushdowns.)
- (c) Determine whether inclusion and equivalence of two PDA languages are decidable. Give reductions from a standard undecidable problem; do not infer inclusion from equivalence or conversely without an argument.

Exercise 9. [*A unary alphabet*]

- (a) Prove that every unary context-free language is regular. You may use the effective version of Parikh's theorem. Deduce decidability of inclusion and equivalence for unary PDA languages.
- (b) Show that the all-traces language of *any* unary LTS is either $\{\varepsilon, a, \dots, a^n\}$ for some $n \geq 0$, or a^* .
- (c) For unary VASSs, give an effective procedure deciding inclusion and equivalence of all-traces languages. You may use decidability of VASS termination, meaning absence of an infinite run from the initial configuration.
- (d) Explain why (b) does not establish regularity of every unary *accepted language* of an arbitrary infinite-state system.

Exercise 10. [*Regularity and unbounded configurations*] Consider a one-counter VASS with initial configuration $(r, 0)$, final control state f , and transitions

$$r \xrightarrow{a, +1} r, \quad r \xrightarrow{b, -1} f, \quad f \xrightarrow{b, -1} f.$$

- (a) Describe its accepted language and prove that it is not regular.
- (b) Describe its all-traces language and determine whether it is regular.
- (c) Construct a VASS with infinitely many reachable configurations whose accepted language is regular. Explain why unboundedness alone does not decide language regularity.

Exercise 11. [*Deterministic VASSs and regular specifications*] Let V be a VASS and A an NFA.

- (a) Show that $L(V) \subseteq L(A)$ is decidable by a product construction. State whether the required VASS test is reachability or coverability.
- (b) Now let V_1, V_2 be deterministic VASSs. Design a procedure for deciding $L(V_1) \subseteq L(V_2)$. Separate rejection by V_2 caused by a missing transition, an insufficient counter value, or a non-final control state. You may use decidability of VASS reachability to semilinear target sets.
- (c) Deduce decidability of accepted-language equivalence for this deterministic model.

Exercise 12. [$\star$ *A decidability map: BPA, BPP, and VASS*] Complete the following table for the *accepted-language conventions* fixed above. Write D (decidable) or U (undecidable). For each non-elementary result, supply a precise theorem reference and check its acceptance convention. A full proof is not required.

Model	$L(A) \subseteq L(B)$	$L(A) = L(B)$	Is $L(A)$ regular?
NFA			
PDA			
BPA			
BPP			
VASS			
Deterministic VASS			

- (a) Investigate which table entries change when accepted languages are replaced by all finite trace languages. State each convention explicitly.